UNIVERSITÀ DELLA CALABRIA

Algorithmic Game Theory — a.a. 2026/2027 — Sebastiano A. Piccolo

Appunti in italiano, parte di `main.pdf` — 5 novembre 2026

LEZIONE 5

Giochi coalizionali

TEORIA

Il percorso della lezione:

1. introduzione ai giochi coalizionali;
2. come rappresentarli;
3. quali coalizioni si formano;
4. come dividere il payoff: **stabilità** (core) e **equità** (nucleolo, valore di Shapley).

5.1 Dai giochi non cooperativi a quelli coalizionali

- I giochi non cooperativi modellano giocatori egoisti e in competizione.
- Giochi come il dilemma del prigioniero o l'attraversamento mostrano che la competizione può portare a esiti subottimi.
- Gli equilibri correlati modellano (e rendono possibile) la cooperazione, ma introducono un dispositivo di coordinamento, un *mediatore*.
- Nella realtà, però, le persone formano spontaneamente **coalizioni** e prendono **accordi vincolanti**.

La situazione tipica che si modella con un gioco coalizionale è questa:

- i giocatori possono trarre vantaggio dal formare coalizioni;
- a ogni coalizione è associato un **valore**;
- il valore totale va **distribuito** tra i giocatori.

Servono due scelte di modellazione:

	Domanda	Modello
Coalizioni	il risultato di una coalizione è indipendente dalle altre?	sì → coalizioni singole; no → più coalizioni insieme (giochi in funzione di partizione)
Valore	l'utilità è trasferibile?	sì → un solo numero per coalizione (TU); no → un insieme di vettori di utilità (NTU)

Definizione 5.1 (Utilità trasferibile e non trasferibile). L'utilità è **trasferibile** (TU) se è la stessa per tutti i membri della coalizione e si può ridistribuire tra loro: i payoff sono monetari (o convertibili in denaro), i giocatori possono farsi pagamenti laterali, e la ridistribuzione non cambia il valore totale. Esempi: dividere profitti, ripartire costi, assegnare seggi in parlamento.
L'utilità è **non trasferibile** (NTU) se ogni membro ha la propria e la coalizione non può convertirla o ridistribuirla liberamente. Esempi: il prestigio di un articolo scientifico scritto insieme, i benefici personali di una collaborazione, i vantaggi legati a indicatori di prestazione individuali.

Attenzione (denaro non vuol dire utilità trasferibile). Il notebook lo dice bene: poter trasferire *denaro* non implica che sia trasferibile l'*utilità*. Un giocatore ricco e uno povero possono trarre utilità molto diverse dalla stessa somma (lezione 2: la funzione di Bernoulli è concava). Assumere TU significa assumere che tutti valutino il denaro allo stesso modo, cioè utilità lineari nel denaro.

5.2 Giochi in funzione caratteristica

Definizione 5.2 (Gioco in funzione caratteristica). Un gioco in funzione caratteristica è una coppia (N, v) dove:

- $N = \{1, \dots, n\}$ è l'insieme dei giocatori, detto anche **grande coalizione**;
- $v : 2^N \to \mathbb{R}$ è la **funzione caratteristica**: per ogni sottoinsieme C di giocatori, $v(C)$ è quanto possono guadagnare i membri di C lavorando insieme. L'utilità $v(C)$ è trasferibile.

Di solito si assume che v sia

- **normalizzata**: $v(\emptyset) = 0$;
- **non negativa**: $v(C) \geq 0$ per ogni $C \subseteq N$;
- **monotona**: $v(C) \leq v(D)$ se $C \subseteq D$.

Esempio (il gelato). Tre persone vogliono comprare del gelato e possono mettere insieme i soldi.

Giocatore	Soldi	Vaschetta	Prezzo
A	\$6	500 g	\$7
B	\$3	750 g	\$9
C	\$3	1000 g	\$11

Il valore di una coalizione è la quantità massima di gelato che può comprare:

$$\begin{aligned} v(\emptyset) = v(\{A\}) = v(\{B\}) = v(\{C\}) &= 0 && \text{(nessuno ha \$7 da solo)} \\ v(\{A,B\}) = v(\{A,C\}) &= 750 && \text{(\$9: la vaschetta da 750 g)} \\ v(\{B,C\}) &= 0 && \text{(\$6: non basta per nulla)} \\ v(\{A,B,C\}) &= 1000 && \text{(\$12: la vaschetta da 1000 g).} \end{aligned}$$

Oltre la funzione caratteristica. I giochi in funzione caratteristica modellano utilità trasferibile e risultati di una coalizione indipendenti da ciò che fanno gli altri. Se i risultati dipendono dalle altre coalizioni si usano i **giochi in funzione di partizione**, con valori $v(C, P)$ dove C è una coalizione della partizione P; se l'utilità non è trasferibile si usano i **giochi NTU**, dove a ogni coalizione è associato un *insieme* di vettori di utilità. Ogni gioco TU si può rappresentare come un gioco NTU con un continuo di azioni, quindi

$$\text{NTU} \supset \text{TU} \supset \text{giochi in funzione caratteristica;}$$

nella pratica si usano soprattutto questi ultimi, che spesso si chiamano semplicemente "giochi TU".

Le due domande. I giochi coalizionali servono a rispondere a due domande:

1. quali coalizioni si formeranno?
2. come si divide il payoff all'interno di una coalizione?

Nel gioco del gelato la coalizione con il benessere più alto è $\{A, B, C\}$. Ma dividere in parti uguali non è equo: A è più decisivo di B e C, perché senza di lui non si compra nulla.

5.3 Struttura di coalizione

Definizione 5.3 (Generazione della struttura di coalizione). Dato un gioco $G = (N, v)$ si cerca la **struttura di coalizione** $CS(G)$, cioè una *partizione* di N in coalizioni disgiunte, che massimizza il benessere sociale $\sum_{C \in CS(G)} v(C)$.

Attenzione (partizione, non ricoprimento). Le slide dicono che una struttura di coalizione è un "ricoprimento" (*cover*) di N. Un ricoprimento può avere insiemi sovrapposti, e un giocatore che sta in due coalizioni verrebbe contato due volte. Quello che si intende, e che le slide stesse scrivono più avanti, è una **partizione**: le coalizioni coprono N e sono a due a due disgiunte.

Esempio (quando non conviene la grande coalizione).

$$\begin{aligned} v(\emptyset) = 0, \quad v(\{A\}) = 0, \quad v(\{B\}) = 300, \quad v(\{C\}) = 0, \\ v(\{A,B\}) = 750, \quad v(\{A,C\}) = 750, \quad v(\{B,C\}) = 400, \quad v(\{A,B,C\}) = 1000. \end{aligned}$$

La struttura di coalizione che massimizza il benessere è $\{\{A, C\}, \{B\}\}$, con $750+300 = 1050 > 1000$: qui la grande coalizione *non* è la scelta migliore.

5.3.1 Il problema è difficile

Le partizioni di un insieme di n elementi sono il **numero di Bell** B_n, che soddisfa

$$\left(\frac{n}{4}\right)^{n/2} \leq B_n < n^n.$$

Enumerarle tutte è impraticabile. Gli algoritmi usati sono la programmazione dinamica, il grafo delle partizioni e le euristiche per giochi strutturati.

5.3.2 Programmazione dinamica

Sia $v^*(S)$ il massimo benessere ottenibile con una struttura di coalizione dei giocatori di S, e $CS^*(S)$ la struttura che lo realizza: $v(CS^*(S)) = v^*(S)$.

Teorema 5.4 (Equazione della programmazione dinamica). Per ogni $C \subseteq N$:

$$v^*(C) = \max\left\{v(C),\ \max_{\substack{C' \cup C'' = C \\ C' \cap C'' = \emptyset}} \left(v^*(C') + v^*(C'')\right)\right\}.$$

O la coalizione C resta intera, o la si divide in due parti, ciascuna delle quali va poi organizzata nel modo migliore.

Dimostrazione. ($\geq$) $v^*(C) \geq v(C)$, perché $\{C\}$ è una struttura di coalizione di C; e $v^*(C) \geq v^*(C') + v^*(C'')$, perché unendo le strutture ottime delle due parti si ottiene una struttura di C. ($\leq$) Se $CS^*(C) = \{C\}$ allora $v^*(C) = v(C)$. Altrimenti sia C' una coalizione di $CS^*(C)$ e $C'' = C \setminus C'$. Il resto, $CS^*(C) \setminus \{C'\}$, è una struttura di coalizione di C'', quindi $v^*(C'') \geq v^*(C) - v(C')$; inoltre $v^*(C') \geq v(C')$. Sommando, $v^*(C') + v^*(C'') \geq v^*(C)$. □

Esempio (la tabella della programmazione dinamica). Con la funzione caratteristica dell'esempio precedente, procedendo per dimensione crescente:

C	$v^*(C)$	$CS^*(C)$
$\{A\}$	0	$\{A\}$
$\{B\}$	300	$\{B\}$
$\{C\}$	0	$\{C\}$
$\{A,B\}$	$\max(v^*(A) + v^*(B),\ v(AB)) = \max(300, 750) = 750$	$\{A,B\}$
$\{A,C\}$	$\max(0, 750) = 750$	$\{A,C\}$
$\{B,C\}$	$\max(300, 400) = 400$	$\{B,C\}$
$\{A,B,C\}$	$\max\big(1000,\ 750+0,\ 750+300,\ 400+0\big) = 1050$	$\{\{A,C\},\{B\}\}$

Per $\{A,B,C\}$ le divisioni in due parti sono tre: $\{A,B\}|\{C\}$, $\{A,C\}|\{B\}$, $\{B,C\}|\{A\}$. La migliore è la seconda.

Attenzione (un errore nella tabella delle slide). Le slide scrivono $v^*(\{B,C\}) = \max(300, 400) = 750$ e riportano 750 in tabella. Il massimo è 400. L'errore non cambia il risultato finale (1050 viene dalla divisione $\{A,C\}|\{B\}$, che non usa $v^*(\{B,C\})$), ma nel passo successivo le slide elencano due

volte la divisione $\{A, B\}|\{C\}$ e mai $\{B, C\}|\{A\}$, proprio quella che userebbe il valore sbagliato. Il codice del laboratorio conferma la tabella corretta.

L'approccio è **esatto**, ma il tempo è $O(3^n)$: non è praticabile per $n \geq 20$.

Approfondimento (da dove viene 3^n). Si considerano tutte le coppie (coalizione C, sua parte C'). Ogni giocatore, in una coppia, può stare in C', in $C \setminus C'$ o fuori da C: tre possibilità per ciascuno degli n giocatori, cioè 3^n coppie. Con $n = 20$ sono circa 3,5 miliardi.

Grafo delle partizioni. Le partizioni si possono disporre su livelli, dal livello 1 (la sola grande coalizione) al livello n (tutti singoli); due partizioni sono collegate se una si ottiene dall'altra fondendo due coalizioni. Le slide mostrano il grafo per 4 giocatori senza svilupparlo; l'idea degli algoritmi che lo usano è esplorare per primi pochi livelli scelti bene e fermarsi quando si ha una garanzia sufficiente sulla distanza dall'ottimo.

In pratica. Si usano euristiche e rappresentazioni compatte della funzione caratteristica: funzioni caratteristiche analitiche, algoritmi di clustering (formazione delle coalizioni per similarità o distanza, partizionamento di grafi), metaeuristiche (algoritmi genetici, *simulated annealing*, . . .).

5.4 Dividere il payoff

Definizione 5.5 (Esito di un gioco TU). Un **esito** di un gioco TU $G = (N, v)$ è una coppia (CS, x) dove:

- $CS = (C_1, \ldots, C_k)$ è una struttura di coalizione, cioè una partizione di N: $\bigcup_i C_i = N$, $C_i \cap C_j = \emptyset$ per $i \neq j$;
- $x = (x_1, \ldots, x_n)$ è un **vettore di payoff** che distribuisce il valore di ogni coalizione di CS: $\sum_{i \in C} x_i = v(C)$ per ogni $C \in CS$ (**efficienza**).

Un esito è un'**imputazione** se soddisfa la **razionalità individuale**: $x_i \geq v(\{i\})$ per ogni $i \in N$.

Esempio (esiti). Con $v(\{1, 2, 3\}) = 9$ e $v(\{4, 5\}) = 4$:

- $\big((\{1, 2, 3\}, \{4, 5\}),\ (3, 3, 3, 3, 1)\big)$ è un esito: $3 + 3 + 3 = 9$, $3 + 1 = 4$;
- $\big((\{1, 2, 3\}, \{4, 5\}),\ (2, 3, 2, 3, 3)\big)$ *non* lo è: la prima coalizione distribuisce 7 e la seconda 6. Non sono ammessi trasferimenti *tra* coalizioni.

5.4.1 Superadditività

Definizione 5.6 (Gioco superadditivo). Un gioco (N, v) è **superadditivo** se

$$v(C_1 \cup C_2) \geq v(C_1) + v(C_2) \qquad \forall C_1, C_2 \subset N \text{ con } C_1 \cap C_2 = \emptyset.$$

Il gioco del gelato è superadditivo: per esempio $v(\{A, B\}) = 750 \geq v(\{A\}) + v(\{B\}) = 0$, $v(\{B, C\}) = 0 \geq 0 + 0$, $v(\{A, B, C\}) = 1000 \geq 750 + 0$.

Nei giochi superadditivi due coalizioni possono sempre fondersi senza perdere nulla: si può assumere che si formi la **grande coalizione**. Non c'è il problema della struttura di coalizione e la notazione si semplifica: un esito è solo un vettore $x = (x_1, \dots, x_n)$ con $\sum_{i \in N} x_i = v(N)$.

> **Osservazione (un'avvertenza delle slide).** Molti articoli definiscono gli esiti in questo modo, come semplici vettori con $\sum x_i = v(N)$, *anche* quando il gioco non è superadditivo. Va tenuto presente quando si leggono: nel gioco della programmazione dinamica, per esempio, dividere $v(N) = 1000$ sarebbe sbagliato, perché la struttura migliore produce 1050.

Da qui in poi si considerano solo giochi TU superadditivi.

5.5 Stabilità: il core

Nel gioco del gelato si deve dividere 1000. Se la divisione è $(200, 200, 600)$, A e B possono comprare da soli la vaschetta da 750 g e dividersela a metà, 375 a testa: meglio di 200. L'esito $(200, 200, 600)$ **non è stabile**.

> **Definizione 5.7 (Core).** Il **core** di un gioco è l'insieme degli esiti stabili, quelli da cui nessuna coalizione vuole sganciarsi:
>
> $$\mathrm{Core}(G) = \left\{ x \;\middle|\; x_i \geq 0 \; \forall i \in N,\ \sum_{i \in N} x_i = v(N),\ \sum_{i \in C} x_i \geq v(C) \; \forall C \subseteq N \right\}.$$
>
> Ogni coalizione riceve almeno quanto guadagnerebbe da sola.

> **Esempio (esiti nel core).**
>
> - Con $v(\{1,2,3\}) = 9$, $v(\{4,5\}) = 4$ e $v(\{2,4\}) = 7$, l'esito $(3,3,3,3,1)$ *non* è nel core: i giocatori 2 e 4 ricevono $3 + 3 = 6 < 7$ e preferiscono formare la coalizione $\{2,4\}$.
> - Nel gelato, $(200, 200, 600)$ non è nel core perché $v(\{A,B\}) = 750 > x_A + x_B = 400$.
> - Nel gelato, $(500, 250, 250)$ *è* nel core: $x_A + x_B = 750 \geq 750$, $x_A + x_C = 750 \geq 750$, $x_B + x_C = 500 \geq 0$. Nessuna deviazione dà di più.

> **Osservazione (il primo esempio).** L'esempio con $(3,3,3,3,1)$ usa una struttura di coalizione $\{\{1,2,3\},\{4,5\}\}$, mentre la definizione di core appena data è quella per la grande coalizione. Il concetto è lo stesso (nessuna coalizione deve poter fare meglio da sola), ma nella versione con strutture di coalizione l'efficienza si chiede coalizione per coalizione.

> **Approfondimento (com'è fatto il core del gelato).** I vincoli che contano sono due. Da $x_A + x_B \geq 750$ e $x_A + x_B + x_C = 1000$ segue $x_C \leq 250$; analogamente $x_B \leq 250$. Il core è quindi
>
> $$\mathrm{Core} = \{x \geq 0 : x_B \leq 250,\ x_C \leq 250,\ x_A = 1000 - x_B - x_C\},$$
>
> e in particolare $x_A \geq 500$ in ogni esito stabile. Si vede bene il potere di A: se B chiedesse più di 250, A potrebbe accordarsi con C e dividere 750 dando a entrambi più di quanto ricevono; lo stesso vale a parti invertite. B e C sono in concorrenza tra loro per essere il partner di A.

5.5.1 Due cattive notizie

Esempio (il core può essere vuoto).

$$v(\{1\}) = v(\{2\}) = v(\{3\}) = 0, \quad v(\{1,2\}) = 7,\ v(\{1,3\}) = 6,\ v(\{2,3\}) = 5, \quad v(\{1,2,3\}) = 8.$$

Per stare nel core serve stabilità, $x_1 + x_2 \geq 7$, $x_1 + x_3 \geq 6$, $x_2 + x_3 \geq 5$, ed efficienza, $x_1 + x_2 + x_3 = 8$. Sommando le tre disuguaglianze: $2(x_1 + x_2 + x_3) \geq 18$, cioè $x_1 + x_2 + x_3 \geq 9 > 8$. Impossibile: il core è vuoto. E il gioco è superadditivo, quindi la superadditività non basta a garantire un core non vuoto.

Esempio (il core può contenere esiti iniqui). Nel gelato $(1000, 0, 0)$ è nel core: $x_A + x_B = 1000 \geq 750$, $x_A + x_C = 1000 \geq 750$, $x_B + x_C = 0 \geq 0$, e la somma è 1000. B e C mettono metà dei soldi e non ricevono nulla, ma nessuna coalizione può fare meglio da sola. **Stabilità non vuol dire equità.**

5.6 Il nucleolo

Definizione 5.8 (Eccesso). L'**eccesso** di una coalizione S rispetto all'esito x è

$$e(S, x) = v(S) - \sum_{i \in S} x_i.$$

Misura l'**insoddisfazione** della coalizione: quanto potrebbe guadagnare da sola rispetto a quello che riceve. Un esito è nel core se e solo se tutti gli eccessi sono ≤ 0.

Esempio (eccessi). Con $v(\{i\}) = 0$, $v(\{i, j\}) = 1$ per ogni coppia e $v(\{1,2,3\}) = 3$:

- $x = (0, 0, 3)$: $e(\{1,2\}, x) = 1 - 0 = 1$, la coppia $\{1,2\}$ è insoddisfatta;
- $x = (1, 2, 0)$: $e(\{1,2\}, x) = 1 - 3 = -2$, la coppia è più che soddisfatta.

Definizione 5.9 (Nucleolo). Sia $\theta(x)$ il **vettore degli eccessi** di tutte le coalizioni (diverse da $\emptyset$ e da N), ordinato dal più grande al più piccolo. Il **nucleolo** è l'imputazione

$$\eta \in X(G) \ : \ \nexists\, y \in X(G) \text{ con } \theta(y) \prec_{lex} \theta(\eta),$$

dove $X(G)$ è l'insieme delle imputazioni e $\prec_{lex}$ l'ordine lessicografico.

In parole: è l'esito che rende **lessicograficamente minimo** il vettore delle insoddisfazioni. Prima si rende la coalizione più insoddisfatta il meno insoddisfatta possibile; a parità, la seconda; e così via.

Esempio (nucleolo del gioco simmetrico). Nel gioco precedente:

- $x = (1, 2, 0)$: $\theta(x) = (0, 0, -1, -1, -2, -2)$;
- $\eta = (1, 1, 1)$: $\theta(\eta) = (-1, -1, -1, -1, -1, -1)$.

Il primo elemento di $\theta(\eta)$ è $-1 < 0$: η è lessicograficamente migliore, ed è il nucleolo (lo conferma il calcolo con l'algoritmo seguente).

Approfondimento (il nucleolo esiste sempre ed è unico). Il nucleolo esiste ed è unico per ogni gioco con almeno un'imputazione. E se il core non è vuoto il nucleolo sta nel core: un esito del core ha tutti gli eccessi ≤ 0, quindi il nucleolo, che minimizza il più grande, ne ha anch'esso il più grande ≤ 0. È per questo che lo si considera la versione "più centrale" del core.

5.6.1 L'algoritmo di Kopelowitz

Il vettore degli eccessi si minimizza lessicograficamente con una *sequenza* di programmi lineari. Il primo minimizza l'eccesso massimo ϵ_1:

$$\begin{aligned} & \min \epsilon_1 \\ LP_1: \quad & \text{s.t. } v(S) - \textstyle\sum_{i\in S} x_i \leq \epsilon_1 && \forall S \subset N,\ S \notin W_0 = \{\emptyset\} \\ & \quad x_i \geq v(\{i\}), \quad \textstyle\sum_{i\in N} x_i = v(N). \end{aligned}$$

Le coalizioni che all'ottimo hanno eccesso esattamente ϵ_1^* formano l'insieme W_1 e il loro eccesso viene *fissato*. Il programma k-esimo minimizza il massimo eccesso delle coalizioni rimaste:

$$\begin{aligned} & \min \epsilon_k \\ LP_k: \quad & \text{s.t. } v(S) - \textstyle\sum_{i\in S} x_i = \epsilon_r^* && \forall S \in W_r,\ r = 1, \ldots, k-1 \\ & \quad v(S) - \textstyle\sum_{i\in S} x_i \leq \epsilon_k && \forall S \notin W_0 \cup W_1 \cup \cdots \cup W_{k-1} \\ & \quad x_i \geq v(\{i\}), \quad \textstyle\sum_{i\in N} x_i = v(N), \end{aligned}$$

con $W_r = \{S \mid e(S,x) = \epsilon_r^*\}$. Ci si ferma quando tutte le coalizioni hanno l'eccesso fissato.

- Ogni programma ha $O(2^n)$ vincoli, uno per coalizione.
- Nel caso peggiore si fissa una coalizione per volta: fino a 2^n iterazioni.
- In generale è **molto poco pratico**.

Attenzione (quali coalizioni fissare). "Le coalizioni con eccesso pari a ϵ_k^*" va inteso come "le coalizioni il cui eccesso vale ϵ_k^* in *tutte* le soluzioni ottime del programma k". Una soluzione ottima qualsiasi può avere per caso altre coalizioni con eccesso ϵ_k^* che in un'altra soluzione ottima potrebbero scendere: fissarle porterebbe a un risultato sbagliato. Nell'implementazione del laboratorio lo si controlla con un programma lineare in più per ciascuna coalizione candidata (si prova ad abbassarne l'eccesso restando all'ottimo).

5.6.2 Il nucleolo può essere iniquo

Esempio (un gioco in cui conta solo A).

$$v(\emptyset) = v(\{A\}) = v(\{B\}) = v(\{C\}) = 0,$$
$$v(\{A,B\}) = v(\{A,C\}) = 750, \quad v(\{B,C\}) = 0, \quad v(\{A,B,C\}) = 750.$$

Il nucleolo è $\eta = (750, 0, 0)$: assegna tutto ad A, anche se almeno uno tra B e C è necessario per ottenere qualcosa. Anche qui stabilità non implica equità.

Attenzione (il vettore degli eccessi nelle slide). Le slide riportano $\theta(\eta) = (0,0,0,0,0,0)$. Gli eccessi di $\eta = (750,0,0)$ sono: $e(\{B\}) = e(\{C\}) = 0$, $e(\{A,B\}) = e(\{A,C\}) = 750 - 750 = 0$, $e(\{B,C\}) = 0$, ma $e(\{A\}) = 0 - 750 = -750$. Quindi $\theta(\eta) = (0,0,0,0,0,-750)$. Il nucleolo resta quello: il core di questo gioco contiene *solo* $(750,0,0)$ (da $x_A + x_B \geq 750$ e $x_A + x_B + x_C = 750$ segue $x_C = 0$, e lo stesso per x_B), e il nucleolo sta nel core.

5.7 Equità: il valore di Shapley

Idea: un esito equo dovrebbe ricompensare ogni giocatore in proporzione al **contributo** che porta agli altri. Se il giocatore i entra nella coalizione C, il suo **contributo marginale** è il surplus che crea:

$$v(C \cup \{i\}) - v(C).$$

Nel gioco precedente, per esempio, $v(\{A,B\}) - v(\{A\}) = 750$ e $v(\{A,B\}) - v(\{B\}) = 750$: il contributo di un giocatore dipende da chi c'è già. Un esito equo si ottiene premiando ciascuno con il suo contributo *medio*.

Definizione 5.10 (Valore di Shapley). Il valore di Shapley considera tutti gli ordini in cui la grande coalizione si può formare e dà a ciascuno la stessa importanza:

$$\phi(i,v) = \frac{1}{|N|!} \sum_{\pi \in \Pi_N} \Big(v\big(B(\pi,i) \cup \{i\}\big) - v\big(B(\pi,i)\big) \Big),$$

dove Π_N è l'insieme delle permutazioni di N e $B(\pi,i)$ l'insieme dei giocatori che precedono i nella permutazione π.

Esempio (Shapley nel gioco del gelato). Contributi marginali di A nei sei ordini:

Ordine	Contributo di A	
A,B,C	$v(\{A\}) - v(\emptyset)$	$= 0$
A,C,B	$v(\{A\}) - v(\emptyset)$	$= 0$
B,A,C	$v(\{A,B\}) - v(\{B\})$	$= 750$
B,C,A	$v(\{A,B,C\}) - v(\{B,C\})$	$= 1000$
C,A,B	$v(\{A,C\}) - v(\{C\})$	$= 750$
C,B,A	$v(\{A,B,C\}) - v(\{B,C\})$	$= 1000$

$$\phi(A) = \tfrac{1}{6}(0 + 0 + 750 + 1000 + 750 + 1000) = 583{,}\overline{3}.$$

Per B i contributi sono $750, 250, 0, 0, 250, 0$, quindi $\phi(B) = \frac{1250}{6} = 208{,}\overline{3}$, e per simmetria $\phi(C) = 208{,}\overline{3}$. Somma: 1000.

Esempio (Shapley nel gioco in cui conta solo A).

Ordine	contributo di A	contributo di B	contributo di C
A, B, C	0	750	0
A, C, B	0	0	750
B, A, C	750	0	0
B, C, A	750	0	0
C, A, B	750	0	0
C, B, A	750	0	0
Valore di Shapley	500	125	125

A differenza del nucleolo, B e C ricevono qualcosa: chi arriva subito dopo A è quello che "sblocca" il valore.

Approfondimento (equo, ma non stabile). $(500, 125, 125)$ non è nel core di questo gioco: il core contiene solo $(750, 0, 0)$, e infatti $x_A + x_B = 625 < 750 = v(\{A, B\})$. A e B da soli prenderebbero di più. Equità e stabilità sono richieste diverse e possono essere incompatibili. Nel gioco del gelato, invece, il valore di Shapley (583,3; 208,3; 208,3) *è* nel core (rispetta $x_B, x_C \leq 250$). Il risultato è coerente con l'esempio delle tre imprese della lezione 1, dove con valore 100 si otteneva (66,67; 16,67; 16,67): è lo stesso gioco in scala.

5.7.1 Caratterizzazione assiomatica

Teorema 5.11 (Shapley). Il valore di Shapley è l'**unico** schema di ripartizione che soddisfa:

1. **efficienza**: $\phi_1 + \phi_2 + \cdots + \phi_n = v(N)$;
2. **giocatore nullo**: se $v(C \cup \{i\}) = v(C)$ per ogni C, allora $\phi_i = 0$;
3. **simmetria**: se i e j sono intercambiabili ($v(C \cup \{i\}) = v(C \cup \{j\})$ per ogni C che non li contiene), allora $\phi_i = \phi_j$;
4. **additività**: $\phi_i(G_1 + G_2) = \phi_i(G_1) + \phi_i(G_2)$.

Inoltre si può calcolare con la formula sui sottoinsiemi:

$$\phi(i, v) = \sum_{\substack{C \subseteq N \\ i \in C}} \frac{(|N| - |C|)!\,(|C| - 1)!}{|N|!} \big(v(C) - v(C \setminus \{i\})\big).$$

Osservazione (da dove viene il coefficiente). Negli ordini in cui i entra trovando già esattamente la coalizione $C \setminus \{i\}$, i $|C| - 1$ predecessori possono stare in $(|C| - 1)!$ ordini e i $|N| - |C|$ successori in $(|N| - |C|)!$. Il coefficiente è la frazione di permutazioni in cui i dà proprio il contributo $v(C) - v(C \setminus \{i\})$. Si passa da $n!$ permutazioni a 2^{n-1} coalizioni contenenti i: è la formula da usare nel codice.

Attenzione ("giocatore fittizio" e "giocatore nullo"). Le slide chiamano l'assioma 2 *dummy player* e chiedono $\phi_i = 0$. In molti testi il *dummy* è un giocatore che aggiunge sempre esattamente

$v(\{i\})$, e riceve $\phi_i = v(\{i\})$; quello che riceve zero è il *null player*, che non aggiunge mai nulla. Con $v(\{i\}) = 0$ le due nozioni coincidono. Le slide scrivono anche “Shapely” invece di Shapley.

5.8 Laboratorio: Core e valore di Shapley

LAB

[CoalitionalGames_Frozensets_blank.ipynb, Lab_5-Coalitional_Game_Theory_Core_Shapley.pdf]

Obiettivo: rappresentare un gioco coalizionale in Python, verificare la superadditività, controllare se un esito sta nel core e calcolare il valore di Shapley. Il notebook contiene gli output attesi; tutti sono riprodotti. In più sono stati implementati e verificati la programmazione dinamica per la struttura di coalizione e il nucleolo.

5.8.1 Rappresentare la funzione caratteristica

Si usa un dizionario indicizzato da `frozenset`: i `frozenset` sono insiemi immutabili, quindi si possono usare come chiavi, e hanno tutte le operazioni degli insiemi (appartenenza, inclusione, unione, intersezione).

```python
from itertools import combinations
from math import factorial

v = {
    frozenset(['A']): 0, frozenset(['B']): 0, frozenset(['C']): 0,
    frozenset(['A', 'B']): 750, frozenset(['A', 'C']): 750,
    frozenset(['B', 'C']): 0,
    frozenset(['A', 'B', 'C']): 1000,
}

def powerset(players):   # tutte le coalizioni NON vuote: 2^n - 1
    return [frozenset(c) for k in range(1, len(players) + 1)
            for c in combinations(players, k)]
```

> **Attenzione (la coalizione vuota non c'è).** Il dizionario del notebook non contiene $\emptyset$, e la `powerset` fornita lo esclude (il commento dice "2^{n-1} frozen sets", ma sono $2^n - 1$). Ogni formula che usa $v(\emptyset)$, come la formula di Shapley con $C = \{i\}$, deve trattarlo a parte: `v.get(S, 0)` invece di `v[S]`, altrimenti un `KeyError`.

5.8.2 Superadditività

```python
def is_superadditive(v):
    for C1 in v:
        for C2 in v:
            if not (C1 & C2):                       # disgiunte
                if v.get(C1 | C2, 0) < v[C1] + v[C2]:
                    return False
    return True
```

Il gioco del gelato dà `True`; il gioco del notebook con $v(A) = 6$, $v(B) = 12$, $v(A, B) = 12$ dà `False` (perché $12 < 6 + 12$). Entrambi come atteso.

5.8.3 Core

```python
def is_stable(x, v, tol=1e-9):
    return all(sum(x[i] for i in C) >= v[C] - tol for C in v)

def is_in_the_core(x, v, tol=1e-9):
    N = max(v)                                          # la grande coalizione
    return (all(xi >= -tol for xi in x.values())
            and abs(sum(x.values()) - v[N]) <= tol
            and is_stable(x, v, tol))
```

Esito	Nel core?	Perché
(333,3334; 333,3333; 333,3333)	no	$x_A + x_B \approx 666{,}7 < 750$
(500; 250; 250)	sì	tutti i vincoli rispettati
(1000; 0; 0)	sì	stabile, anche se iniquo

Attenzione (`max(v)` trova la grande coalizione, ma per un motivo sottile). Il notebook ricava la grande coalizione con `max(characteristic_function)`. Funziona, ma non perché confronti le dimensioni: per i `frozenset` l'operatore `>` significa "sovrainsieme proprio". `max` tiene il primo elemento e lo sostituisce con ogni elemento che lo contiene strettamente; la grande coalizione contiene tutte le altre, quindi appena la incontra non la lascia più. Se la grande coalizione mancasse dal dizionario, `max` restituirebbe una coalizione qualsiasi senza segnalare nulla.

Osservazione (perché non si calcola il core). Il notebook si limita a *verificare* se un esito sta nel core. Calcolarlo tutto richiederebbe di trovare i vertici del poliedro definito dai vincoli, con una sequenza di programmi lineari. Per sapere solo se il core è *vuoto* basta però un programma lineare: si cerca un x qualsiasi che soddisfi i vincoli. Applicato al gioco con $v(\{1,2\}) = 7$, $v(\{1,3\}) = 6$, $v(\{2,3\}) = 5$, $v(N) = 8$ il risolutore conferma che non esiste.

5.8.4 Valore di Shapley

```python
def shapley_value(player, v):
    N = max(v)
    n = len(N)
    phi = 0.0
    for C in v:
        if player in C:
            s = len(C)
            weight = factorial(n - s) * factorial(s - 1) / factorial(n)
            phi += weight * (v[C] - v.get(C - {player}, 0))
    return phi

def shapley(v):
    return {p: shapley_value(p, v) for p in max(v)}
```

Risultato sul gelato: $\{A : 583{,}33;\ B : 208{,}33;\ C : 208{,}33\}$, come atteso. Una seconda implementazione con le permutazioni dà gli stessi valori.

Sottoinsiemi o permutazioni. Il notebook raccomanda la formula sui sottoinsiemi: $O(2^n)$ invece di $O(n!)$. Su giochi casuali con n giocatori, tempi misurati per calcolare il valore di *tutti* i giocatori:

n	Sottoinsiemi	Permutazioni
6	0,1 ms	3,8 ms
8	0,8 ms	304 ms
9	1,9 ms	3 409 ms
10	5,5 ms	3 628 800 ordini, non eseguito

I valori calcolati con i due metodi coincidono ogni volta. Il codice sopra scorre tutte le coalizioni per ogni giocatore, quindi il costo totale è $n \cdot 2^n$: comunque esponenziale. Per giochi con molti giocatori si usano approssimazioni (per esempio campionando permutazioni a caso) o rappresentazioni compatte, come anticipa la fine del notebook.

> **Attenzione (“il valore di Shapley è individualmente razionale”).** Il notebook elenca tra le proprietà $\phi(i, v) \geq v(\{i\})$. Vale per i giochi *superadditivi*: lì ogni contributo marginale è $v(C \cup \{i\}) - v(C) \geq v(\{i\})$, e quindi lo è anche la media. Senza superadditività può fallire. Il notebook stesso ricorda subito dopo l’altro limite: il valore di Shapley non è necessariamente nel core.

5.8.5 In più: struttura di coalizione e nucleolo

Programmazione dinamica. Implementazione diretta dell’equazione della teoria, coalizione per coalizione in ordine di dimensione:

```python
def coalition_structure_dp(v):
    N = max(v)
    best, struct = {}, {}
    for C in sorted(powerset(sorted(N)), key=len):
        best[C], struct[C] = v[C], [C]
        members = sorted(C)
        for k in range(1, len(C)):
            for part in combinations(members, k):
                C1 = frozenset(part); C2 = C - C1
                if min(C1) != min(C):          # ogni divisione una volta sola
                    continue
                if best[C1] + best[C2] > best[C]:
                    best[C] = best[C1] + best[C2]
                    struct[C] = struct[C1] + struct[C2]
    return best, struct
```

Sul gioco dell’esempio restituisce esattamente la tabella corretta: $v^*(\{B, C\}) = 400$ e $v^*(N) = 1050$ con $CS^* = \{\{A, C\}, \{B\}\}$.

Nucleolo. Implementato con l’algoritmo di Kopelowitz e `linprog`, fissando a ogni passo solo le coalizioni il cui eccesso non si può abbassare. Risultati:

Gioco	Nucleolo	Vettore degli eccessi
Simmetrico ($v(ij) = 1$, $v(N) = 3$)	$(1, 1, 1)$	$(-1, -1, -1, -1, -1, -1)$
Conta solo A ($v(N) = 750$)	$(750, 0, 0)$	$(0, 0, 0, 0, 0, -750)$
Gelato ($v(N) = 1000$)	$(750, 125, 125)$	$(-125, -125, -125, -125, -250, -750)$

Nel gelato nucleolo e Shapley sono diversi: $(750; 125; 125)$ contro $(583{,}3; 208{,}3; 208{,}3)$. Entrambi stanno nel core, ma il nucleolo "bilancia" le insoddisfazioni (le coppie $\{A, B\}$ e $\{A, C\}$ e i singoli B e C hanno tutti eccesso -125), mentre Shapley premia i contributi marginali.

Riepilogo della lezione

- Gioco in funzione caratteristica (N, v): $v(C)$ è quanto guadagna la coalizione C; utilità trasferibile.
- Struttura di coalizione: partizione di N che massimizza $\sum v(C)$. Le partizioni sono il numero di Bell; la programmazione dinamica la trova in $O(3^n)$.
- Superadditività: fondersi non fa mai perdere, quindi si forma la grande coalizione e un esito è un vettore x con $\sum x_i = v(N)$.
- **Core**: esiti in cui ogni coalizione riceve almeno $v(C)$. Può essere vuoto e può contenere esiti iniqui.
- **Nucleolo**: minimizza lessicograficamente gli eccessi $e(S, x) = v(S) - x(S)$; esiste, è unico, sta nel core se questo non è vuoto; si calcola con una sequenza di programmi lineari (Kopelowitz). Può essere iniquo.
- **Shapley**: contributo marginale medio su tutti gli ordini di ingresso; unico a soddisfare efficienza, giocatore nullo, simmetria, additività. Equo ma non necessariamente stabile.
- **Lab**: `frozenset` come chiavi, superadditività, verifica del core, Shapley sui sottoinsiemi ($n \cdot 2^n$ contro $n!$); in più programmazione dinamica e nucleolo.